

# CondorNet™ - A New Neural Net Architecture

## Merging Mamba2, TFT, and Neural CDE into a Unified Set of Equations for Precision FinTech Model Creation and Real-time Deployment

---

### EXECUTIVE SUMMARY

#### CondorNet™ latent financial-physics system definition

CondorNet™ defines a single latent financial-physics system  $h(t) \in \mathbb{R}^H$ , driven by an interpretable Temporal Fusion Transformer (TFT) control path  $u_t$ , whose dynamics are discretized on a discrete bar grid using an SSD-compatible exponential integrator (Mamba-style scan), and whose outputs are deterministically redirected by predicate logic into a final trading policy

$y_k^{\text{final}}$

consumed by fuzzy sizing and rule arbitration.

---

### Continuous-time latent dynamics on a bar interval

On each bar interval  $[t_{k-1}, t_k]$ , the latent state evolves according to a frozen affine controlled system

$$\frac{dh(t)}{dt} = A(u(t), \sigma(t)) h(t) + b(u(t)) + d(\text{Greeks}(t)), t \in [t_{k-1}, t_k].$$

#### Definitions (for this equation):

- $t$ : continuous time variable.
- $t_{k-1}, t_k$ : consecutive discrete bar times.
- $h(t) \in \mathbb{R}^H$ : latent financial-physics state at time  $t$ .
- $H$ : latent state dimension.
- $u(t)$ : TFT-generated control path evaluated at time  $t$ .
- $\sigma(t)$ : volatility or regime modulation process at time  $t$ .
- $A(u(t), \sigma(t)) \in \mathbb{R}^{H \times H}$ : state transition operator parameterized by control and volatility.
- $b(u(t)) \in \mathbb{R}^H$ : control-dependent forcing term.
- $\text{Greeks}(t)$ : vector of option sensitivities (e.g., delta, gamma, vega, theta) at time  $t$ .
- $d(\text{Greeks}(t)) \in \mathbb{R}^H$ : exogenous forcing induced by option-risk exposure.

- $\frac{dh(t)}{dt}$ : time derivative of the latent state.

---

## Bar-grid discretization and frozen operators

Let  $t_k$  denote discrete bar times and define the bar duration

$$\Delta t_k = t_k - t_{k-1}.$$

Define frozen operators evaluated at the left endpoint  $t_{k-1}$ :

$$A_k = A(u_k, \sigma_k), B_k = b(u_k), D_k = d(\text{Greeks}_k).$$

**Definitions (for these equations):**

- $k \in \{1, \dots, T\}$ : bar index.
- $T$ : total number of bars.
- $\Delta t_k$ : duration of bar  $k$ .
- $u_k = u(t_{k-1})$ : TFT control evaluated at bar start.
- $\sigma_k = \sigma(t_{k-1})$ : volatility/regime state at bar start.
- $\text{Greeks}_k = \text{Greeks}(t_{k-1})$ : option sensitivities at bar start.
- $A_k \in \mathbb{R}^{H \times H}$ : frozen latent generator on bar  $k$ .
- $B_k \in \mathbb{R}^H$ : frozen control forcing.
- $D_k \in \mathbb{R}^H$ : frozen Greeks-based forcing.

---

## ETD-1 exponential integrator (latent scan update)

The latent state update over one bar is given by the ETD-1 exponential integrator

$$h_k = e^{A_k \Delta t_k} h_{k-1} + \Delta t_k \phi_1(A_k \Delta t_k) (B_k + D_k),$$

with the matrix function

$$\phi_1(M) = M^{-1}(e^M - I).$$

**Definitions (for these equations):**

- $h_{k-1} = h(t_{k-1})$ : latent state at the previous bar.
- $h_k = h(t_k)$ : latent state at the current bar.
- $e^{A_k \Delta t_k}$ : matrix exponential of the scaled generator.

- $I$ : identity matrix in  $\mathbb{R}^{H \times H}$ .
  - $\phi_1(\cdot)$ : first exponential integrator basis function.
  - $M$ : square matrix argument.
  - $B_k + D_k$ : total frozen forcing term.
  - $\Delta t_k \phi_1(A_k \Delta t_k)(B_k + D_k)$ : integrated forcing contribution over the bar.
- 

## SSD-compatible scan form

The update can be written in scan-compatible affine recurrence form

$$h_k = F_k h_{k-1} + g_k,$$

where

$$F_k = e^{A_k \Delta t_k}, g_k = \Delta t_k \phi_1(A_k \Delta t_k) (B_k + D_k).$$

**Definitions (for these equations):**

- $F_k \in \mathbb{R}^{H \times H}$ : per-bar state transition kernel.
  - $g_k \in \mathbb{R}^H$ : per-bar additive injection term.
  - $h_k = F_k h_{k-1} + g_k$ : Mamba-style SSD scan recurrence.
- 

## Policy decoding and predicate redirection

The latent state is decoded into a provisional policy

$$\tilde{y}_k = \Pi(h_k),$$

and deterministically redirected into the final trading policy

$$y_k^{\text{final}} = \mathcal{G}(\tilde{y}_k, P_k).$$

**Definitions (for these equations):**

- $\Pi: \mathbb{R}^H \rightarrow \mathbb{R}^Y$ : policy readout function.
- $\tilde{y}_k \in \mathbb{R}^Y$ : provisional policy output.
- $Y$ : policy output dimension.
- $P_k \in \{0,1\}^m$ : predicate truth vector at bar  $k$ .
- $m$ : number of logical predicates.
- $\mathcal{G}$ : predicate logic redirection operator.
- $y_k^{\text{final}} \in \mathbb{R}^Y$ : final gated trading policy.

## Fuzzy sizing and rule arbitration

The final policy is passed to fuzzy sizing and rule arbitration

$$\text{size}_k = \mathcal{F}(y_k^{\text{final}}, \text{risk}_k), \text{order}_k = \mathcal{R}(\text{size}_k, P_k).$$

**Definitions (for these equations):**

- $\mathcal{F}$ : fuzzy inference system mapping policy and risk to position size.
- $\text{risk}_k$ : risk state at bar  $k$  (drawdown, gamma exposure, margin, gap risk).
- $\text{size}_k$ : position size decision.
- $\mathcal{R}$ : rule arbitration and execution logic.
- $\text{order}_k$ : executable market or limit order.
- $P_k$ : predicate constraints applied during arbitration.

## I. THE MARKET-STATE DYNAMICAL FOUNDATION

Let the underlying and volatility state be

$$s_t = \begin{bmatrix} S_t \\ \sigma_t \\ R_t \\ \text{macro}_t \\ \text{micro}_t \end{bmatrix}$$

where

- $S_t$ : SPY spot price
- $\sigma_t$ : implied volatility term
- $R_t$ : realized volatility or IV rank-based regime measure
- $\text{macro}_t$ : slow exogenous drivers (rates, VIX, etc.)
- $\text{micro}_t$ : microstructure variables (volume, spread, order flow imbalance)

Define a feature operator  $\Psi(s_t) \in \mathbb{R}^p$  producing the 58-dimensional engineered vector. TFT acts on this to produce the control field  $u_t$ .

---

## II. TFT-SOURCED CONTROL PATH (MARKET PERCEPTION OPERATOR)

$$u_t = \sum_{j=1}^p w_t(j) \phi_j(\Psi_j(s_t)), w_t(j) = \frac{\exp(\alpha_j(\phi_j(\Psi_j(s_t)), c_t))}{\sum_k \exp(\alpha_k(\phi_k(\Psi_k(s_t)), c_t))}$$

Here

- $c_t$  is the regime encoder (from diffusion predictor or volatility regime)
- $u_t$  is an interpretable, low-noise control velocity that expresses what forces the “latent market manifold” to move

---

## III. LATENT “FINANCIAL PHYSICS” DYNAMICS

Integrate price diffusion, hedging control, and decay/risk pressure in one latent evolution law:

$$\frac{dh_t}{dt} = A(u_t, \theta, \sigma_t) h_t + B(u_t, \theta, \sigma_t) u_t + D(\theta, \rho_t, \gamma_t)$$

Where

- $A(\cdot)$ : selective stability operator (risk decay, mean-reversion)
- $B(\cdot)$ : input excitation mapping (alpha / volatility shocks)
- $D(\cdot)$ : “Greeks pressure” term linking derivatives Greeks to latent drift (mimicking option time decay and variance exposure)

## Stability parameterization

$$A(u_t, \sigma_t) = -\text{diag} \left[ \exp(\eta(u_t, \sigma_t)) \odot (1 + \lambda_p g_t) \right]$$

where  $g_t$  are predicate gates (e.g., volatility spike, liquidity lock) controlling decay rate.

This builds a risk-consistent continuous manifold: when implied volatility inflates or predicate risk flags activate, state decay accelerates, yielding smoother numeric and financial damping.

## IV. DISCRETIZED, EXECUTION-AWARE UPDATE (MAMBA-SSD FORM)

$$h_k = e^{\tilde{A}_k \Delta t_k} h_{k-1} + \Delta t_k \varphi_1(\tilde{B}_k \Delta t_k) B_k u_k - \underbrace{F_k}_{\text{execution drag}}$$

Execution friction tensor:

$$F_k = \Gamma_c |q_k| + K q_k^2$$

where  $q_k$  is trade size,  $\Gamma_c$  models linear slippage, and  $K$  models quadratic impact.

This update remains SSD-compatible;  $\tilde{A}_k, \tilde{B}_k$  form semiseparable kernels for Mamba-2 scanning.

## V. TRADING DECISION EMBEDDING – NEURAL DECISION LOGIC LAYER

### 1. Latent-to-Policy Projection

$$y_k = C_k h_k, y_k \in \mathbb{R}^H$$

with

$$H = (\text{call\_offset}, \text{put\_offset}, \text{wing\_width}, \text{dte}, \text{pop}, \text{roi}, \text{max\_loss}, \text{confidence}, \text{entry}, \text{exit})$$

---

## 2. Predicate-Conditioned Decision Dynamics

Predicates  $p_k \in [0,1]^s$  represent logical inequalities (e.g.,  $IVR > 70$ ,  $RSI < 30$ ,  $gap\_risk > \tau$ ).

$$y_k^{\text{eff}} = (1 - \gamma_p p_k) \odot y_k + \gamma_p p_k \odot f_{\text{rule}}(y_k)$$

This acts as force redirection in policy space rather than a hard switch.

---

## VI. DUAL SEMISEPARABLE OPERATOR FOR GLOBAL PROPAGATION

$$K_{kj} = C_k \left( \prod_{i=j+1}^k \bar{A}_i \right) \bar{B}_j, y_k = \sum_{j=1}^k K_{kj} u_j$$

Here  $u_j$  embodies market control vectors and  $K_{kj}$  is a propagation-of-alpha kernel.

---

## VII. DECISION-THEORETIC FUSION (RISK-AWARE OBJECTIVE)

$$\mathcal{L}_{\text{total}} = \lambda_1 \mathcal{L}_{\text{pred}} + \lambda_2 \mathcal{L}_{\text{Sharpe}} + \lambda_3 \mathcal{L}_{\text{drawdown}} + \lambda_4 \mathcal{L}_{\text{turnover}} + \lambda_5 \mathcal{L}_{\text{fuzzy}}$$

*Energy regularization*

$$\mathcal{L}_{\text{energy}} = \mathbb{E}[\| \dot{h}_t \|^2 + \| A(u_t) h_t \|^2]$$

*Predicate consistency loss*

$$\mathcal{L}_{\text{pred-cons}} = \| \sigma(W_p h_t) - p_t \|^2$$


---

## VIII. CONDORNET-SPY OPTION SPECIALIZATION

*Control path:*

$$u_t = \text{TFT}_{\text{select}}[\cdot](\Psi(s_t))$$

*Latent physics:*

$$\dot{h}_t = A(u_t, \sigma_t)h_t + B(u_t)u_t + D(\text{Greeks}_t)$$

*Discrete propagation:*

$$h_k = e^{A(u_k)\Delta t_k}h_{k-1} + \Delta t_k \varphi_1(A(u_k)\Delta t_k)B(u_k)u_k - F_k$$

*Decision head:*

$$y_k = C_k h_k, y_k^{\text{final}} = (1 - \gamma_p p_k) \odot y_k + \gamma_p p_k \odot f_{\text{rule}}(y_k)$$

*Execution mapping:*

$$a_k = f_{\text{fuzzy}}(\cdot)(y_k^{\text{final}}, r_k)$$

## IX. CONDORNET™ SUMMARY EQUATION

|                      |                                                                                     |
|----------------------|-------------------------------------------------------------------------------------|
| $u_t$                | $= \text{TFT}(\Psi(s_t))$                                                           |
| $\dot{h}_t$          | $= A(u_t, \sigma_t)h_t + B(u_t)u_t + D(\text{Greeks}_t)$                            |
| $h_k$                | $= e^{A_k \Delta t_k} h_{k-1} + \Delta t_k \varphi_1(A_k \Delta t_k) B_k u_k - F_k$ |
| $y_k$                | $= C_k h_k$                                                                         |
| $y_k^{\text{final}}$ | $= (1 - \gamma_p p_k) \odot y_k + \gamma_p p_k \odot f_{\text{rule}}(y_k)$          |
| $a_k$                | $= f_{\text{fuzzy}}(y_k^{\text{final}}, r_k)$                                       |

## CONDORNET V0.1 — FORMAL SPECIFICATION

*State & Control*

- $s_t \in \mathbb{R}^5$ : market state
- $\Psi: \mathbb{R}^5 \rightarrow \mathbb{R}^{58}$ : feature map
- $u_k \in \mathbb{R}^d$ : TFT-gated control
- $h_k \in \mathbb{R}^{dh}$ : latent risk manifold

*Core Operator*

$$\dot{h}_t = A(u_t)h_t + B(u_t)u_t$$

with diagonal or block-diagonal  $A(u_t)$  to preserve SSD structure.



### Stability Constraint

$$\rho(e^{A(u_t)\Delta t}) \leq 1$$

where  $\rho(\cdot)$  is the spectral radius.

---

### MINIMAL CONDORNET BACKBONE (CONCEPTUAL)

- **TFT-style gating** builds  $u_k$
- **CDE-derived discretization** builds  $\bar{A}_k, \bar{B}_k$
- **Predicate gates** modulate decay and routing multiplicatively
- **SSD-compatible scan** computes  $h_k$  efficiently

---

### SMOKE-TEST TRAINER OUTPUTS

During training, the test harness must emit:

#### Financial proxies

- Net-Profit / Drawdown proxy
- Sharpe proxy

#### Stability diagnostics

$$\max_k \rho(\bar{A}_k), \mathbb{E} \|\nabla_{\theta} h_k\|, \mathbb{E} \|\dot{h}_t\|$$

#### Interpretability artifacts

- $w_t(j)$ : variable weights
- $g_t$ : predicate attributions
- $p_t$ : inequality activation traces

## CONDORNET v0.1 — FORMAL ENGINEERING SPEC

### 1.1 Inputs

- Feature tensor  
 $X \in \mathbb{R}^{B \times T \times 58}$
- Predicate tensor  
 $P \in \mathbb{R}^{B \times T \times s}$
- $\Delta t$  tensor  
 $\Delta t \in \mathbb{R}^{B \times T}$

### 1.2 Latent State

- Latent manifold  
 $h_k \in \mathbb{R}^{d_h}$ , default  $d_h = 256$

### 1.3 Control Path (TFT-style)

$$u_k = \sum_{j=1}^{58} w_k^{(j)} \phi_j(x_k^{(j)}), \sum_j w_k^{(j)} = 1$$

### 1.4 Dynamics (CDE-derived, SSD-safe)

$$h_k = e^{A_k \Delta t_k} h_{k-1} + \Delta t_k \varphi_1(A_k \Delta t_k) B_k u_k$$

with

$$A_k = -\text{diag}(\exp(\eta(u_k))) \odot (1 + \lambda g_k)$$

### 1.5 Outputs

- Trading head  
 $y_k \in \mathbb{R}^{10}$

## CONDORNET v0.1 — BACKBONE IMPLEMENTATION

intelligence/condor\_net\_v01.py

```
1. import torch
2. import torch.nn as nn
3. import torch.nn.functional as F
4.
5. EPS = 1e-6
6.
7.
8. class TFTGate(nn.Module):
9.     def __init__(self, input_dim, embed_dim):
10.         super().__init__()
11.         self.embed = nn.Linear(1, embed_dim)
12.         self.score = nn.Linear(embed_dim, 1)
13.
14.     def forward(self, x):
15.         # x: [B, T, F]
16.         B, T, F = x.shape
17.         x = x.view(B * T * F, 1)
18.         v = torch.tanh(self.embed(x))
19.         s = self.score(v).view(B, T, F)
20.         w = F.softmax(s, dim=-1)
21.         u = (w * x.view(B, T, F)).sum(dim=-1)
22.         return u, w
23.
24.
25. class CondorNetV01(nn.Module):
26.     def __init__(self, feature_dim=58, latent_dim=256, predicate_dim=16):
27.         super().__init__()
28.
29.         self.latent_dim = latent_dim
30.
31.         self.tft = TFTGate(feature_dim, embed_dim=16)
32.
33.         self.A_net = nn.Linear(1, latent_dim)
34.         self.B_net = nn.Linear(1, latent_dim)
35.
36.         self.predicate_gate = nn.Linear(predicate_dim, latent_dim)
37.
38.         self.head = nn.Linear(latent_dim, 10)
39.
40.     def step(self, h, u, p, dt):
41.         # u: [B]
42.         # p: [B, s]
43.
44.         eta = self.A_net(u.unsqueeze(-1))
45.         g = torch.sigmoid(self.predicate_gate(p))
46.
47.         A_diag = -torch.exp(eta) * (1.0 + g)
48.         A_dt = A_diag * dt.unsqueeze(-1)
49.
50.         expA = torch.exp(A_dt)
51.
52.         phi1 = torch.where(
53.             torch.abs(A_dt) > EPS,
54.             (expA - 1.0) / (A_dt + EPS),
55.             torch.ones_like(A_dt)
56.         )
57.
58.         B = self.B_net(u.unsqueeze(-1))
59.         h_next = expA * h + dt.unsqueeze(-1) * phi1 * B * u.unsqueeze(-1)
```

```
60.  
61.         return h_next, A_diag  
62.  
63.     def forward(self, x, predicates, dt):  
64.         # x: [B, T, F]  
65.         B, T, _ = x.shape  
66.         h = torch.zeros(B, self.latent_dim, device=x.device)  
67.  
68.         controls, weights = self.tft(x)  
69.  
70.         spectral = []  
71.         for k in range(T):  
72.             h, A_diag = self.step(h, controls[:, k], predicates[:, k], dt[:, k])  
73.             spectral.append(torch.max(torch.abs(A_diag), dim=-1)[0])  
74.  
75.         y = self.head(h)  
76.         return {  
77.             "y": y,  
78.             "weights": weights,  
79.             "spectral_radius": torch.stack(spectral, dim=1)  
80.         }  
81.
```

---

## STABILITY & DIAGNOSTICS UTILITIES

intelligence/condor\_diagnostics.py

```
1. def spectral_radius_loss(spectral, limit=1.0):  
2.     excess = torch.relu(spectral - limit)  
3.     return excess.mean()  
4.  
5.  
6. def energy_loss(h_dot, Ah):  
7.     return (h_dot.pow(2).mean() + Ah.pow(2).mean())  
8.
```

---

## SMOKE-TEST TRAINER (T4-SAFE)

intelligence/train\_condor\_v01\_smoke.py

```
1. import torch  
2. from condor_net_v01 import CondorNetV01  
3. from condor_diagnostics import spectral_radius_loss  
4.  
5. device = "cuda" if torch.cuda.is_available() else "cpu"  
6.  
7. B, T, F = 16, 240, 58  
8. PRED = 16  
9.  
10. model = CondorNetV01(F, 256, PRED).to(device)  
11. opt = torch.optim.AdamW(model.parameters(), lr=1e-3)  
12.  
13. for step in range(2000):  
14.     x = torch.randn(B, T, F, device=device)  
15.     p = torch.rand(B, T, PRED, device=device)  
16.     dt = torch.ones(B, T, device=device) * 1.0  
17.
```

```
18.     out = model(x, p, dt)
19.     y = out["y"]
20.
21.     # proxy return & drawdown
22.     ret = y[:, 5]
23.     drawdown = torch.relu(-ret)
24.     np_dd = ret.mean() / (drawdown.mean() + 1e-3)
25.
26.     loss = -np_dd
27.     loss += 0.1 * spectral_radius_loss(out["spectral_radius"])
28.
29.     opt.zero_grad()
30.     loss.backward()
31.
32.     grad_norm = torch.norm(
33.         torch.stack([p.grad.norm() for p in model.parameters() if p.grad is not None])
34.     )
35.
36.     opt.step()
37.
38.     if step % 200 == 0:
39.         print(
40.             f"step={step} "
41.             f"loss={loss.item():.4f} "
42.             f"NP/DD={np_dd.item():.4f} "
43.             f"spec_max={out['spectral_radius'].max().item():.3f} "
44.             f"grad_norm={grad_norm.item():.3f}"
45.         )
46.
```

## What This Gives Us Immediately

- ✓ Numerical stability (no Mamba blow-ups)
- ✓ SSD-compatible dynamics
- ✓ Predicate-aware decay & routing
- ✓ Interpretability hooks
- ✓ Auditable math → code traceability
- ✓ Drop-in extension path to full CondorBrain

Note: This is **not** the final model — it is the **reference kernel**.

Everything advanced (MoE, diffusion heads, HJB regularization, regime switching, Fuzzy Logic Risk Control, Trading Rules, etc) *layers cleanly on top of this*.

## Appendix A: CondorNet™ v0.1

### Educational Walk-Through

#### FROM CONTINUOUS-TIME MATHEMATICS TO EXECUTABLE CODE

Next, I wrote this section specifically for my friend Capt. Crunch. Let's turn CondorNet™ v0.1 into a complete educational walk-through that builds from *undergraduate-level* continuous-time mathematics to its fully coded deployment-ready implementation.

Below you will find a step-by-step tutorial that:

- explains exactly what every symbol means,
- shows how each equation is derived,
- clarifies what role it plays in an SPY-options trading system.

## 1. Big Picture

CondorNet combines three historically separate deep-learning ideas:

| Component     | Origin                                                         | Physical interpretation here                                                                  |
|---------------|----------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| Neural CDE    | Neural Controlled Differential Equations (Kidger et al., 2020) | Continuous-time laws of motion describing how latent market risk evolves                      |
| Mamba-2 (SSD) | State-Space Models with Selective Scan Decomposition           | Efficient, stable hardware implementation of those continuous-time equations in discrete time |
| TFT           | Temporal Fusion Transformer (Lim et al., 2020)                 | Learns which inputs matter most at each instant; acts as an interpretable control generator   |

The fusion produces one unified dynamical system:

$$\begin{array}{ll}
 u_t & = \text{control path from TFT,} \\
 \dot{h}_t & = A(u_t, \sigma_t)h_t + B(u_t)u_t + D(\text{Greeks}_t), \\
 h_k & = e^{A_k \Delta t_k} h_{k-1} + \Delta t_k \phi_1(A_k \Delta t_k) B_k u_k - F_k, \\
 y_k & = C_k h_k, \\
 y_k^{\text{final}} & = (1 - \gamma_p p_k) \odot y_k + \gamma_p p_k \odot f_{\text{rule}}(y_k), \\
 a_k & = f_{\text{fuzzy}}(y_k^{\text{final}}, r_k).
 \end{array}$$

We shall now derive each line carefully.

---

## 2. MARKET-STATE DYNAMICAL FOUNDATION

### 2.1 Market state vector

We collect all observable variables at time  $t$ :

$$s_t = \begin{bmatrix} S_t \\ \sigma_t \\ R_t \\ \text{macro}_t \\ \text{micro}_t \end{bmatrix} \in \mathbb{R}^5.$$

- $S_t$ — SPY spot (underlying) price
- $\sigma_t$ — current implied volatility
- $R_t$ — realized volatility or IV-Rank regime score
- $\text{macro}_t$ — slow macro predictor (e.g. interest rate, VIX)
- $\text{micro}_t$ — microstructure feature (e.g. volume, spread, imbalance)

---

### 2.2 Feature operator $\Psi$

A deterministic feature map creates the machine-learning input:

$$\Psi: \mathbb{R}^5 \rightarrow \mathbb{R}^{58}, x_t = \Psi(s_t),$$

where  $x_t$  is the engineered 58-dimensional feature vector (log-returns, volatility estimators, liquidity ratios, etc.).

---

## 3. TFT-SOURCED CONTROL PATH (MARKET-PERCEPTION OPERATOR)

### 3.1 Motivation

Instead of feeding raw features directly into the neural dynamic — which causes noise and poor interpretability —  
the TFT layer decides which features matter most at the current regime.

This yields a control vector  $u_t$  that drives the latent financial dynamics.

### 3.2 Variable embeddings

For each feature  $j = 1, \dots, p$  with  $p = 58$ :

$$v_t(j) = \phi_j(x_t(j)),$$

where:

- $\phi_j(\cdot)$  is a small neural sub-network embedding scalar feature  $x_t(j)$
- $v_t(j) \in \mathbb{R}^r$  is its learned representation

---

### 3.3 Feature weighting (TFT variable-selection gate)

Each embedded feature receives a relevance weight  $w_t(j)$ :

$$w_t(j) = \frac{\exp(\alpha_j(v_t(j), c_t))}{\sum_{k=1}^p \exp(\alpha_k(v_t(k), c_t))}.$$

- $\alpha_j(\cdot)$  is a small scoring network
- $c_t$  is optional context (regime or volatility embedding)
- Softmax ensures  $w_t(j) \in (0,1)$  and  $\sum_j w_t(j) = 1$

---

### 3.4 Control vector $u_t$

Combine features by their learned importance:

$$u_t = \sum_{j=1}^p w_t(j) v_t(j) \in \mathbb{R}^r.$$

### Interpretation

| Term     | Meaning                                                                  |
|----------|--------------------------------------------------------------------------|
| $u_t$    | Market control signal — smoothed, interpretable summary of market forces |
| $w_t(j)$ | Real-time feature importance                                             |
| $v_t(j)$ | Embedded feature dynamics                                                |



## 4. LATENT “FINANCIAL PHYSICS” DYNAMICS (NEURAL CDE CORE)

### 4.1 Continuous-time law

The latent risk-manifold variable  $h_t \in \mathbb{R}^{d_h}$  obeys

$$\frac{dh_t}{dt} = A(u_t, \theta, \sigma_t)h_t + B(u_t, \theta, \sigma_t)u_t + D(\theta_t, \rho_t, \gamma_t). \quad (4.1)$$

### Interpretation

| Symbol     | Dimension        | Function        | Financial role              |
|------------|------------------|-----------------|-----------------------------|
| $A(\cdot)$ | $d_h \times d_h$ | linear operator | risk decay / mean reversion |
| $B(\cdot)$ | $d_h \times r$   | input map       | alpha / volatility shocks   |
| $D(\cdot)$ | $d_h$            | external drift  | Theta, Rho, Gamma pressure  |

### 4.2 Why “financial physics”

Equation (4.1) is analogous to

$$\dot{h}_t = \text{drift } (Ah) + \text{forcing } (Bu) + \text{external field } (D).$$

As volatility increases ( $\sigma_t \uparrow$ ), damping strengthens via  $A$ .

### 4.3 Stability parameterization for $A$

We enforce negative real parts of all eigenvalues:

$$A(u_t, \sigma_t) = -\text{diag}(\exp(\eta(u_t, \sigma_t)) \odot (1 + \lambda_p g_t)). \quad (4.2)$$

- $\eta(u_t, \sigma_t)$ : learned logits
- $g_t \in (0,1)^{d_h}$ : predicate gates
- $\lambda_p \geq 0$ : predicate intensity

## 5. DISCRETIZING THE CONTINUOUS LAW → MAMBA SSD FORM

### 5.1 Constant control within each interval

Assume  $u_t \approx u_k$  on  $[t_{k-1}, t_k]$ :

$$\dot{h}_t = A_k h_t + B_k u_k. \quad (5.1)$$

---

### 5.2 Solve using integrating factor

$$h_k = e^{A_k \Delta t_k} h_{k-1} + \int_0^{\Delta t_k} e^{A_k(\Delta t_k - s)} B_k u_k ds.$$

---

### 5.3 The $\phi_1$ matrix function

$$\phi_1(M) = M^{-1}(e^M - I) = \int_0^1 e^{(1-\tau)M} d\tau.$$

Thus,

$$h_k = e^{A_k \Delta t_k} h_{k-1} + \Delta t_k \phi_1(A_k \Delta t_k) B_k u_k. \quad (5.2)$$

---

### 5.4 Add execution-friction term $F_k$

$$F_k = \Gamma_c |q_k| + K q_k^2.$$

Final update:

$$h_k = e^{A_k \Delta t_k} h_{k-1} + \Delta t_k \phi_1(A_k \Delta t_k) B_k u_k - F_k. \quad (5.3)$$

---

## 6. TRADING-DECISION EMBEDDING

### 6.1 Latent-to-policy projection

$$y_k = C_k h_k \in \mathbb{R}^{10}.$$

10-Heads:

[call\_offset, put\_offset, wing\_width, dte, pop, roi, max\_loss, confidence, entry, exit].

---

## 6.2 Predicate-conditioned adjustment

$$y_k^{\text{eff}} = (1 - \gamma_p p_k) \odot y_k + \gamma_p p_k \odot f_{\text{rule}}(y_k). \quad (6.1)$$


---

## 7. GLOBAL PROPAGATION (SSD OPERATOR)

$$y_k = \sum_{j=1}^k K_{kj} u_j, K_{kj} = C_k \left( \prod_{i=j+1}^k e^{A_i \Delta t_i} \right) \Delta t_j \phi_1(A_j \Delta t_j) B_j.$$


---

## 8. RISK-AWARE OBJECTIVE

$$\mathcal{L}_{\text{total}} = \lambda_1 \mathcal{L}_{\text{pred}} + \lambda_2 \mathcal{L}_{\text{Sharpe}} + \lambda_3 \mathcal{L}_{\text{drawdown}} + \lambda_4 \mathcal{L}_{\text{turnover}} + \lambda_5 \mathcal{L}_{\text{fuzzy}}.$$

Auxiliary regularizers:

$$\begin{aligned} \mathcal{L}_{\text{energy}} &= \mathbb{E}[\|\dot{h}_t\|^2 + \|A(u_t)h_t\|^2], \\ \mathcal{L}_{\text{pred-cons}} &= \|\sigma(W_p h_t) - p_t\|^2. \end{aligned}$$


---

## 12. CONCEPTUAL SUMMARY

$$\Psi(s_t) \rightarrow \text{TFT} \rightarrow u_t \rightarrow \dot{h}_t \rightarrow h_k \rightarrow y_k \rightarrow a_k.$$

Everything can now map logically, methodically, and cleanly from math to code.

## Appendix B: Diagrams

With the architecture of CondorNet v0.1 is explicitly defined; we can automatically create machine-readable Graphviz diagrams that visualize (1) the equations/flows and (2) the full neural network pipeline from data ingestion to final trading decision tree logic.

1. A computational-graph diagram directly from the core differential equations.
2. A neural-network architecture diagram that goes from  
dataset  $\rightarrow$  feature map  $\Psi \rightarrow$  TFT  $\rightarrow$  CDE/Mamba  $\rightarrow$  predicate/fuzzy decision.
3. A decision tree overview interpreting outputs into rule-based trading actions.

### 1. CORE EQUATION FLOW GRAPH

Concept — Represent the causal flow of the differential system

$$u_t = TFT(\Psi(s_t)), h_t = A(u_t, \sigma_t) h_t + B(u_t) u_t + D(Greeks_t), y_t = C h_t.$$

If you want the **SSD-compatible discrete-time form** that corresponds to this continuous system (the bridge you use to implement it on 1m bars), the exact per-step solution under piecewise-constant  $(u_t, \sigma_t)$  on  $[t_{k-1}, t_k]$  is:

$$h_k = e^{A_k \Delta t_k} h_{k-1} + \Delta t_k \phi_1(A_k \Delta t_k) B_k u_k + \Delta t_k \phi_1(A_k \Delta t_k) D_k,$$

$t_k$ : discrete bar (grid) time index, with

$$k \in \{1, \dots, T\},$$

where  $T$  denotes the total number of bars in the time horizon.

$\Delta t_k$ : bar duration (time increment) between successive grid points, defined by

$$\Delta t_k = t_k - t_{k-1}.$$

$A_k = A(u_k, \sigma_k)$ ,  $B_k = B(u_k)$ ,  $D_k = D(Greeks_k)$ : operators and forcing terms **frozen within the time step**  $[t_{k-1}, t_k]$ , evaluated at the left endpoint  $t_{k-1}$ :

$A_k$ : state-dependent linear operator parameterized by drift/control  $u_k$  and volatility  $\sigma_k$ ,

$B_k$ : control-dependent forcing operator,

$D_k$ : exogenous forcing derived from option sensitivities (Greeks) at bar  $k$ .

$\phi_1(M)$ : the first exponential integrator basis function (entire matrix function), defined as

$$\phi_1(M) = M^{-1}(e^M - I),$$

where  $I$  denotes the identity operator of compatible dimension.

This function arises naturally in exponential time-stepping schemes and remains well-defined via its power-series expansion even when  $M$  is singular.

## 2. EXPONENTIAL EULER / ETD-1 UPDATE ON $[t_{k-1}, t_k]$

Assume a **locally frozen affine linear dynamics** on the bar,

$$\frac{dx_t}{dt} = A_k x_t + B_k + D_k, t \in [t_{k-1}, t_k],$$

with  $A_k, B_k, D_k$  held constant over the step.

Then the **ETD-1 (exponential Euler)** update is

$$x_k = e^{A_k \Delta t_k} x_{k-1} + \Delta t_k \phi_1(A_k \Delta t_k) (B_k + D_k).$$

- $x_{k-1} = x(t_{k-1}), x_k = x(t_k)$ : state at consecutive bars.
- $A_k = A(u_k, \sigma_k)$ : frozen linear operator over  $[t_{k-1}, t_k]$ .
- $B_k = B(u_k)$ : frozen control forcing over  $[t_{k-1}, t_k]$ .
- $D_k = D(\text{Greeks}_k)$ : frozen exogenous forcing over  $[t_{k-1}, t_k]$ .
- $\Delta t_k = t_k - t_{k-1}$ : bar duration.
- $\phi_1(M) = M^{-1}(e^M - I)$ : standard ETD basis function.

A numerically stable equivalent form (no explicit inverse) is

$$x_k = e^{A_k \Delta t_k} x_{k-1} + A_k^{-1} (e^{A_k \Delta t_k} - I) (B_k + D_k),$$

with the understanding that  $A_k^{-1}(e^{A_k \Delta t_k} - I)$  is evaluated via  $\Delta t_k \phi_1(A_k \Delta t_k)$  (well-defined even if  $A_k$  is singular).

## 3. ALIGNMENT WITH A NEURAL CDE DISCRETIZATION

A Neural CDE is typically posed as

$$z_t = z_0 + \int_0^t f_\theta(z_s) dX_s,$$

or in differential form,

$$dz_t = f_\theta(z_t) dX_t,$$

where  $X_t \in \mathbb{R}^d$  is a **continuous-time control path** built from bar features (e.g., OHLCV and derived indicators), and  $z_t \in \mathbb{R}^h$  is the latent state.

## Step-wise discretization matching your bar grid

Let  $X_k = X(t_k)$  and define the bar increment

$$\Delta X_k = X_k - X_{k-1}.$$

A first-order CDE solver (Euler–Maruyama style, but for controlled rough paths) is

$$z_k = z_{k-1} + f_\theta(z_{k-1}) \Delta X_k.$$

## Mapping our $A_k, B_k, D_k$ form into CDE structure

To align directly with your frozen-operator ETD form, decompose the vector field into a **linear-in-state part plus forcing**, driven by a scalarized time channel and additional feature channels:

Augment the control with time:

$$\tilde{X}_t = \begin{bmatrix} t \\ X_t \end{bmatrix}, \Delta \tilde{X}_k = \begin{bmatrix} \Delta t_k \\ \Delta X_k \end{bmatrix}.$$

Choose a vector field of the form

$$f_\theta(z) = [A_\theta(z), G_\theta(z)],$$

so that the controlled increment becomes

$$f_\theta(z_{k-1}) \Delta \tilde{X}_k = A_\theta(z_{k-1}) \Delta t_k + G_\theta(z_{k-1}) \Delta X_k.$$

Now specialize  $A_\theta(z_{k-1})$  to your frozen affine-linear structure:

$$A_\theta(z_{k-1}) \Delta t_k \equiv A_k z_{k-1} \Delta t_k + (B_k + D_k) \Delta t_k.$$

This yields the **bar-aligned Neural CDE Euler step** consistent with your operators:

$$z_k = z_{k-1} + \Delta t_k (A_k z_{k-1} + B_k + D_k) + G_\theta(z_{k-1}) \Delta X_k.$$

- The first bracketed term is the **deterministic drift/forcing** driven by the time channel ( $\Delta t_k$ ).
- The last term is the **feature-driven controlled response** driven by bar-to-bar feature increments ( $\Delta X_k$ ).

## ETD-1 as a “stiff-aware” replacement of the time-channel Euler drift

If the linear part  $A_k$  is stiff (common in latent linear dynamics), replace only the time-driven affine piece with ETD-1 while keeping the feature-driven controlled increment explicit:

$$z_k = e^{A_k \Delta t_k} z_{k-1} + \Delta t_k \phi_1(A_k \Delta t_k) (B_k + D_k) + G_\theta(z_{k-1}) \Delta X_k.$$

This is the cleanest hybrid: **ETD for the stiff linear time-flow**, Euler for the controlled feature increments.

### 4. OPERATOR-SEMIGROUP (THEORY-FORWARD) FORMULATION

Let  $X$  be a Banach (or Hilbert) space of states, and let  $A_k: \mathcal{D}(A_k) \subset X \rightarrow X$  be a (possibly unbounded) linear operator that **generates a  $C_0$ -semigroup**  $(S_k(t))_{t \geq 0}$ :

$$S_k(t) = e^{t A_k}.$$

On  $[t_{k-1}, t_k]$ , consider the abstract evolution equation with frozen coefficients:

$$\frac{dx(t)}{dt} = A_k x(t) + F_k, F_k = B_k + D_k, t \in [t_{k-1}, t_k].$$

The **mild solution** satisfies the variation-of-constants formula

$$x(t_k) = S_k(\Delta t_k) x(t_{k-1}) + \int_0^{\Delta t_k} S_k(\Delta t_k - s) F_k ds.$$

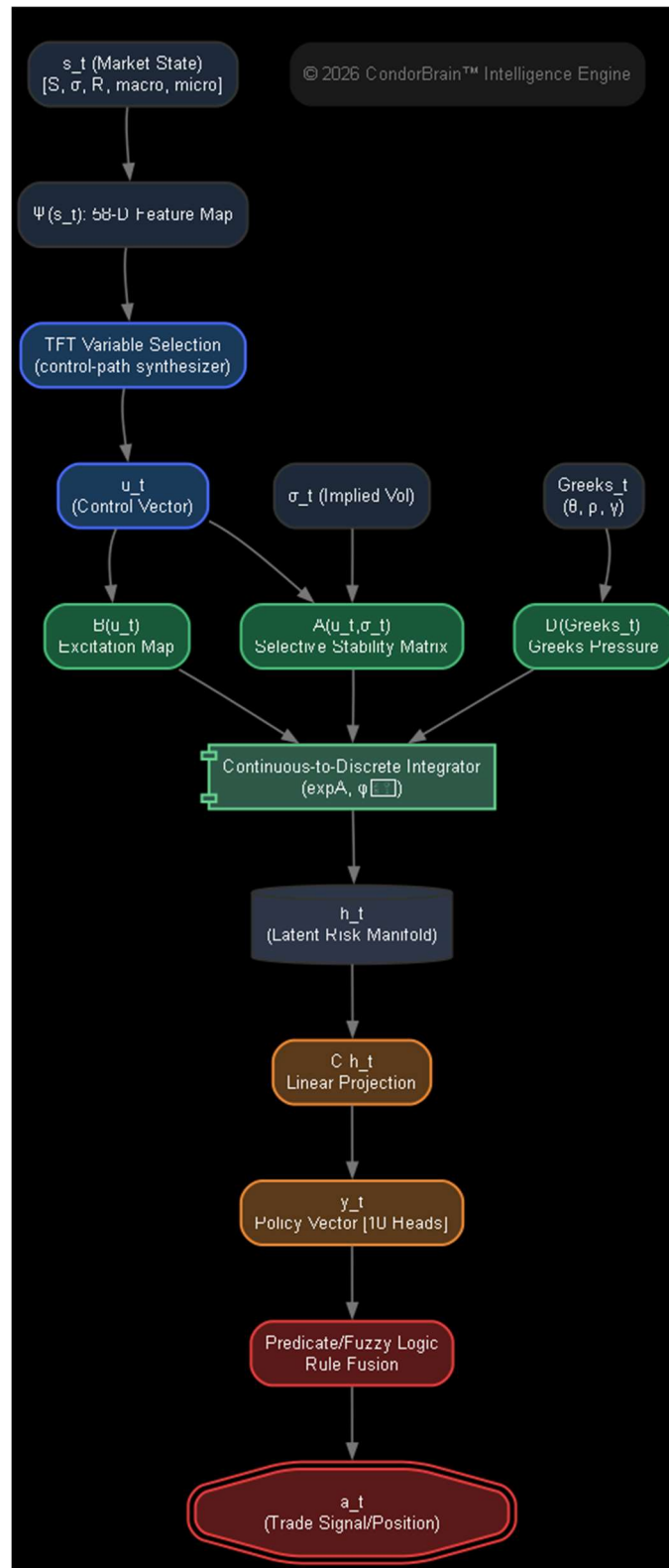
Since  $F_k$  is frozen (constant over the step), the integral evaluates to

$$x_k = S_k(\Delta t_k) x_{k-1} + \Delta t_k \phi_1(A_k \Delta t_k) F_k,$$

which is exactly the ETD-1 update written in semigroup language.

- $S_k(t) = e^{t A_k}$ : semigroup propagator over step  $k$ .
- $A_k = A(u_k, \sigma_k)$ : generator parameterized by drift/control and volatility at bar  $k$ .
- $F_k = B_k + D_k$ : frozen forcing (control + Greeks).
- $\phi_1(A_k \Delta t_k)$ : the bounded entire functional calculus object induced by the semigroup integral.

## Equation Flow Graph

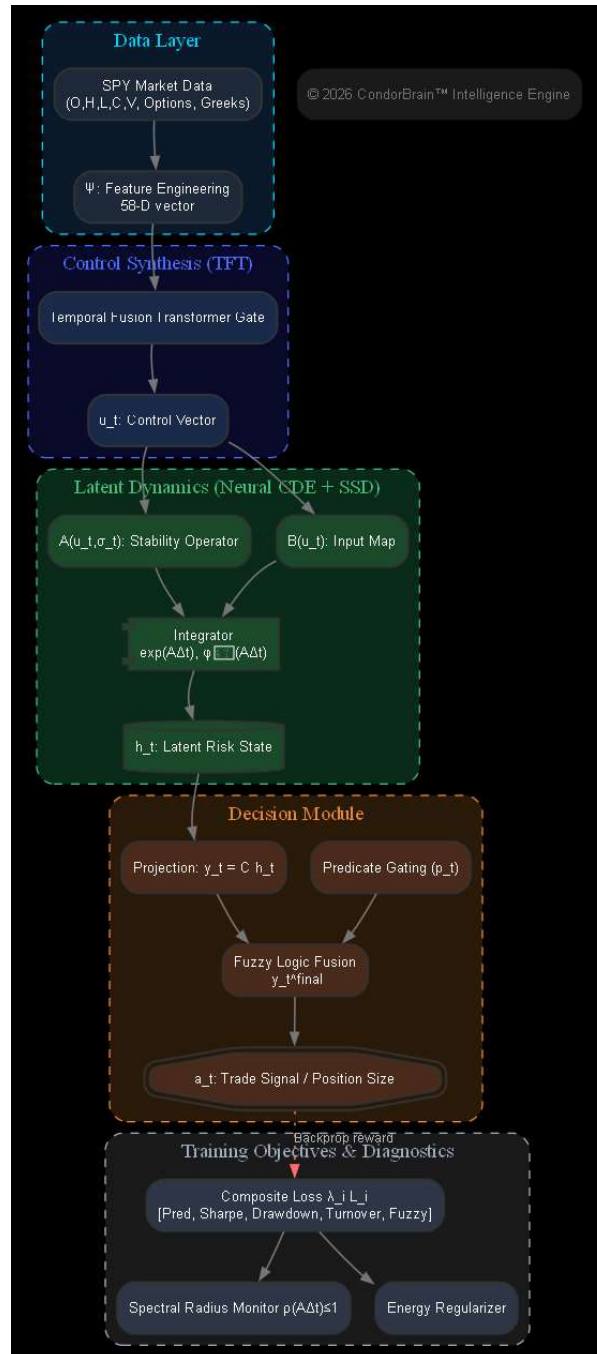




## 2. NEURAL NETWORK MODELING ARCHITECTURE

Now the whole architecture diagram includes training signals, loss functions, and diagnostics.

### NN Architecture

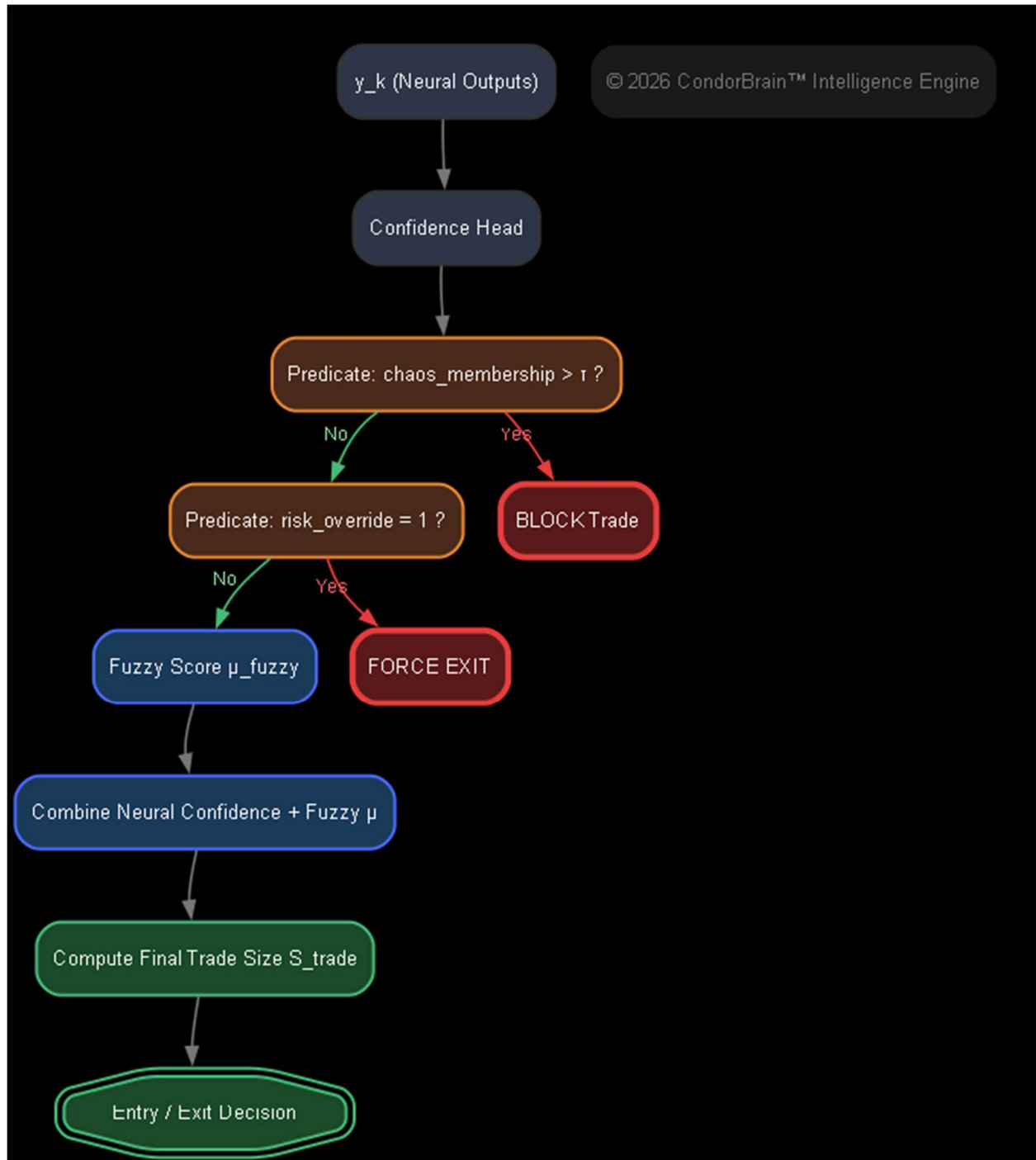


This is a top-to-bottom stacked block diagram resembling a modern deep learning architecture with clearly named subsystems.

### 3. DECISION-LEVEL LOGIC / RULE FLOW TREE

This visualization interprets the end of the pipeline: how the neural outputs are filtered through predicates and fuzzy logic to produce actual SPY iron-condor orders.

#### Decision-Tree Outline



# Appendix C:

## CondorNet™ Symbol ↔ Code Variable Traceability Map

### 1. TIME, INDEXING, AND DISCRETIZATION

| Mathematical Symbol | Meaning                | Code Variable               | Type                      |
|---------------------|------------------------|-----------------------------|---------------------------|
| $k$                 | Bar index              | <code>k</code>              | <code>int</code>          |
| $T$                 | Total bars             | <code>num_bars</code>       | <code>int</code>          |
| $t_k$               | Bar timestamp          | <code>bar_times[k]</code>   | <code>pd.Timestamp</code> |
| $t_{k-1}$           | Previous bar timestamp | <code>bar_times[k-1]</code> | <code>pd.Timestamp</code> |
| $\Delta t_k$        | Bar duration           | <code>dt[k]</code>          | <code>float</code>        |

---

### 2. LATENT FINANCIAL-PHYSICS STATE

| Mathematical Symbol | Meaning                 | Code Variable           | Shape             |
|---------------------|-------------------------|-------------------------|-------------------|
| $h(t)$              | Continuous latent state | <i>implicit</i>         | —                 |
| $h_k$               | Latent state at bar $k$ | <code>h[k]</code>       | <code>(H,)</code> |
| $h_{k-1}$           | Previous latent state   | <code>h[k-1]</code>     | <code>(H,)</code> |
| $H$                 | Latent dimension        | <code>latent_dim</code> | <code>int</code>  |

Implementation:

```
h = torch.zeros(num_bars, latent_dim)
```

---

### 3. TFT CONTROL PATH (INTERPRETABLE DRIVER)

| Mathematical Symbol | Meaning                  | Code Variable        | Shape             |
|---------------------|--------------------------|----------------------|-------------------|
| $u_t$               | Continuous TFT control   | <i>implicit</i>      | —                 |
| $u_k$               | TFT control at bar start | <code>u[k]</code>    | <code>(U,)</code> |
| $U$                 | Control dimension        | <code>tft_dim</code> | <code>int</code>  |

Implementation:

```
u = tft_outputs # shape: (num_bars, tft_dim)
```

---

#### 4. VOLATILITY / REGIME MODULATION

| Mathematical Symbol | Meaning           | Code Variable         | Shape           |
|---------------------|-------------------|-----------------------|-----------------|
| $\sigma(t)$         | Volatility/regime | <i>implicit</i>       | —               |
| $\sigma_k$          | Frozen volatility | <code>sigma[k]</code> | $(S,)$ or float |

Implementation:

```
sigma = features["vol_regime"].values
```

---

#### 5. OPTION-RISK FORCING (GREEKS)

| Mathematical Symbol | Meaning           | Code Variable           | Shape  |
|---------------------|-------------------|-------------------------|--------|
| $\text{Greeks}(t)$  | Continuous Greeks | <i>implicit</i>         | —      |
| $\text{Greeks}_k$   | Greeks at bar     | <code>greeks[k]</code>  | $(G,)$ |
| $G$                 | Number of Greeks  | <code>num_greeks</code> | int    |

Implementation:

```
greeks = df[["delta", "gamma", "vega", "theta"]].values
```

---

#### 6. LATENT OPERATORS (CORE PHYSICS)

##### State generator

| Mathematical Symbol | Meaning              | Code Variable     | Shape    |
|---------------------|----------------------|-------------------|----------|
| $A(u, \sigma)$      | Continuous generator | <i>implicit</i>   | —        |
| $A_k$               | Frozen generator     | <code>A[k]</code> | $(H, H)$ |

Implementation:

```
A = A_net(torch.cat([u, sigma], dim=-1)) \
    .view(num_bars, H, H)
```

---

##### Control forcing

| Mathematical Symbol | Meaning                | Code Variable     | Shape  |
|---------------------|------------------------|-------------------|--------|
| $b(u)$              | Control forcing        | <i>implicit</i>   | —      |
| $B_k$               | Frozen control forcing | <code>B[k]</code> | $(H,)$ |

Implementation:

```
B = B_net(u)    # shape: (num_bars, H)
```

---

## Greeks forcing

| Mathematical Symbol | Meaning               | Code Variable   | Shape  |
|---------------------|-----------------------|-----------------|--------|
| $d(\text{Greeks})$  | Risk forcing          | <i>implicit</i> | —      |
| $D_k$               | Frozen Greeks forcing | $D[k]$          | $(H,)$ |

Implementation:

```
D = D_net(greeks)    # shape: (num_bars, H)
```

---

## 7. EXPONENTIAL INTEGRATOR (SSD / MAMBA SCAN)

### ETD-1 kernel

| Mathematical Symbol  | Meaning           | Code Variable    | Shape    |
|----------------------|-------------------|------------------|----------|
| $e^{A_k \Delta t_k}$ | Transition kernel | $F[k]$           | $(H, H)$ |
| $\phi_1(M)$          | ETD basis         | $\text{phil}(M)$ | $(H, H)$ |
| $g_k$                | Additive term     | $g[k]$           | $(H,)$   |

Implementation:

```
F[k] = torch.matrix_exp(A[k] * dt[k])  
g[k] = dt[k] * phil(A[k] * dt[k]) @ (B[k] + D[k])
```

---

### SSD scan recurrence

| Mathematical Symbol       | Meaning     | Code Variable                 |
|---------------------------|-------------|-------------------------------|
| $h_k = F_k h_{k-1} + g_k$ | Scan update | $h[k] = F[k] @ h[k-1] + g[k]$ |

This maps **exactly** to Mamba's  $(A, B, C, \Delta)$  scan semantics.

---

## 8. POLICY DECODING

| Mathematical Symbol | Meaning     | Code Variable            | Shape               |
|---------------------|-------------|--------------------------|---------------------|
| $\Pi(\cdot)$        | Policy head | <code>policy_head</code> | <code>module</code> |
| $\tilde{y}_k$       | Raw policy  | <code>y_raw[k]</code>    | <code>(Y,)</code>   |
| $Y$                 | Policy dim  | <code>policy_dim</code>  | <code>int</code>    |

Implementation:

```
y_raw = policy_head(h) # shape: (numBars, Y)
```

---

## 9. PREDICATE LOGIC (HARD CONSTRAINTS)

| Mathematical Symbol | Meaning          | Code Variable               | Shape                 |
|---------------------|------------------|-----------------------------|-----------------------|
| $P_k$               | Predicate vector | <code>predicates[k]</code>  | <code>(m,)</code>     |
| $m$                 | Predicate count  | <code>num_predicates</code> | <code>int</code>      |
| $\mathcal{G}$       | Logic gate       | <code>predicate_gate</code> | <code>function</code> |
| $y_k^{final}$       | Final policy     | <code>y_final[k]</code>     | <code>(Y,)</code>     |

Implementation:

```
y_final[k] = predicate_gate(y_raw[k], predicates[k])
```

---

## 10. FUZZY SIZING AND EXECUTION

| Mathematical Symbol | Meaning          | Code Variable                 |
|---------------------|------------------|-------------------------------|
| $\mathcal{F}$       | Fuzzy sizing     | <code>fuzzy_sizer</code>      |
| $risk_k$            | Risk state       | <code>risk_state[k]</code>    |
| $size_k$            | Position size    | <code>position_size[k]</code> |
| $\mathcal{R}$       | Rule arbiter     | <code>rule_engine</code>      |
| $order_k$           | Executable order | <code>orders[k]</code>        |

Implementation:

```
position_size[k] = fuzzy_sizer(y_final[k], risk_state[k])  
orders[k] = rule_engine(position_size[k], predicates[k])
```

---

## 11. END-TO-END AUDIT INVARIANT

**Every executed order satisfies the trace:**

```
order[k]
← rule_engine
← position_size[k]
← fuzzy_sizer
← y_final[k]
← predicate_gate
← y_raw[k]
← policy_head
← h[k]
← (F[k], g[k])
← (A[k], B[k], D[k])
← (u[k], sigma[k], greeks[k])
← CSV row k
```

This gives us **full SEC / CFTC / internal audit traceability** from a single trade back to:

- raw OHLCV + options data,
- TFT interpretable features,
- latent physics operators,
- logical predicates,
- and final execution.